

Adaptive Directional Options Trading via Gradient Boosted Ensemble Classification and Proximal Policy-Optimized Execution

System Architecture, Mathematical Foundations,
and Empirical Validation

Kripta Studios Quantitative Research Division

March 2026

Abstract

We present the complete architecture of an automated intraday trading system for zero-days-to-expiration (0DTE) index options on the S&P 500 (SPX) and NASDAQ-100 (QQQ). The system operates as a two-stage pipeline: (1) a *directional classifier* based on an ensemble of Gradient Boosted Trees (GBT) trained via anchored walk-forward validation emits a probabilistic signal $\hat{y} \in \{\text{SHORT}, \text{HOLD}, \text{LONG}\}$; (2) a *Proximal Policy Optimization* (PPO) agent consumes this signal together with a 175-dimensional state vector to execute option strike selection, entry timing, and exit management with real-time options pricing.

This document provides the mathematical foundations, design rationale, training methodology, reward engineering, and empirical results that justify the architectural evolution from a legacy Multi-Layer Perceptron (MLP) to the current GBT+RL framework. The migration resolved a fundamental over-parameterization failure, recovering a true out-of-sample directional edge of +10.7% above base rate and enabling profitable execution with a profit factor exceeding 1.5 across walk-forward windows spanning 897 trading days.

Contents

1	Problem Formulation	2
1.1	Objective	2
1.2	Market Microstructure	2
1.3	Feature Space	2
2	Stage 1: Directional Classifier (GBT Ensemble)	3
2.1	Architectural Migration: MLP $\rightarrow$ GBT	3
2.1.1	Root Cause: Over-Parameterization	3
2.1.2	Diagnostic Test	4
2.2	Why Gradient Boosted Trees	4
2.3	LightGBM Configuration	4
2.4	Feature Normalization	5

3	Walk-Forward Training Pipeline	5
3.1	Temporal Splitting	5
3.2	Class Balancing	6
3.3	Calibration	6
3.4	Window Selection and Ensemble Construction	6
3.5	Temporal Sample Weighting	7
4	Target Label Engineering	7
4.1	Asymmetric Profit/Stop Labeling	7
4.2	Maximum Move Computation	8
5	Stage 2: RL Execution Agent (PPO)	8
5.1	Motivation	8
5.2	State Space	8
5.3	Action Space	9
5.4	Network Architecture	9
5.4.1	Backbone	9
5.4.2	Action Heads	10
5.4.3	Weight Initialization	10
5.5	PPO Training	10
5.6	Curriculum Learning	11
6	Reward Function Design	11
6.1	Step Reward (Dense)	11
6.2	Sniper Step Reward	12
6.3	Terminal Reward (Sparse)	12
6.3.1	1. Core P&L Reward (Asymmetric)	12
6.3.2	2. Patience Bonus	13
6.3.3	3. Hard Stop Penalty	13
6.3.4	4. Strike Quality Bonus	13
6.3.5	5. Early Exit Penalty	13
6.3.6	6. Under-Capture Penalty	13
6.3.7	7. Entry Timing Bonus	14
7	Risk Management: Hard Exit Rules	14
8	RL Environment: SPXOptionsEnv	14
8.1	Episode Structure	14
8.2	Options Pricing	14
8.3	Stratified Sampling	14
9	System Integration	15
9.1	GBTRL Interface	15
9.2	Data Pipeline	15

10 Backtesting Framework	15
10.1 Dual Simulation	15
10.2 Position Sizing	15
10.3 Drawdown Mitigation	15
10.4 Empirical Backtest Results	16
10.4.1 Per-Ticker Decomposition	16
10.4.2 RL Strike Selection Analysis	16
10.4.3 RL Exit Reason Distribution	17
11 Conclusion	17

1 Problem Formulation

1.1 Objective

The primary objective is to construct a system that, at each 5-minute bar t during U.S. equity market hours (09:30–16:00 ET), produces:

1. A **directional classification** $\hat{y}_t \in \{0, 1, 2\}$ corresponding to $\{\text{SHORT}, \text{HOLD}, \text{LONG}\}$ with an associated confidence $c_t = \max_k p_k(t) \in [0, 1]$.
2. Given a non-HOLD signal with $c_t \geq c_{\min}$, an **execution policy** π_θ that selects the option strike (via delta bucket), times the entry within a sniper observation window, and manages the position until exit.

1.2 Market Microstructure

The system operates on 0DTE SPX options contracts whose expiration falls on the current trading day. These instruments exhibit:

- **Extreme theta decay:** time value vanishes entirely by 16:00 ET, creating an exponentially accelerating cost of holding.
- **High gamma exposure:** near-expiration options exhibit convex payoffs, amplifying directional moves.
- **Wide bid-ask spreads:** deeply OTM strikes can exhibit spreads of 6% of mid price, while ITM spreads compress to $\sim 1\%$.

We model transaction costs via a delta-regime half-spread estimate:

$$\text{HalfSpread}(|\Delta|) = \begin{cases} 0.06 \cdot P_{\text{mid}} & \text{if } |\Delta| < 0.30 \quad (\text{OTM}) \\ 0.03 \cdot P_{\text{mid}} & \text{if } 0.30 \leq |\Delta| \leq 0.60 \quad (\text{ATM}) \\ 0.01 \cdot P_{\text{mid}} & \text{if } |\Delta| > 0.60 \quad (\text{ITM}) \end{cases} \quad (1)$$

1.3 Feature Space

The observation at each timestep consists of $D = 163$ continuously-valued features derived from the underlying spot price and the complete options chain. These are organized into the following categories:

Category	Description	Dims
Spot & Returns	Price, log returns, realized vol	8
Greek Exposures	$\Delta, \Gamma, \theta, \mathcal{V}, \rho$ by expiry	24
GEX / DEX / VEX	Gamma, delta, vanna exposure measures	18
Volume Profiles	Put/call volume, open interest ratios	12
IV Surface	ATM IV, skew, term structure, smile slopes	16

Category	Description	Dims
Order Flow	Net gamma, vanna/charm cross-products	14
Level Proximity	Distance to support/resistance in bps	6
Temporal	Time of day, day of week, minutes to close	5
Cross-Ticker	QQQ context features (correlation, spread)	12
Derived Ratios	Composite signals, z-scores, log-transforms	48
Total		163

All features with extreme-scale distributions (e.g., `delta_gamma_ratio`, `net_zomma`, `gamma_x_near_*`) undergo a sign-preserving log transform during data collection:

$$\tilde{x} = \text{sign}(x) \cdot \log(1 + |x|) \quad (2)$$

This prevents neural saturation and reduces the kurtosis of features with raw values exceeding 10^6 .

2 Stage 1: Directional Classifier (GBT Ensemble)

2.1 Architectural Migration: MLP $\rightarrow$ GBT

The legacy architecture employed a PyTorch-based Multi-Layer Perceptron with approximately 150,000 trainable parameters. Walk-forward validation revealed a catastrophic failure mode: the model achieved 0% precision and recall for out-of-sample directional trades at all confidence thresholds above 0.50.

2.1.1 Root Cause: Over-Parameterization

The fundamental mismatch lay in the model capacity relative to sample size. The training dataset contained approximately 138,000 individual 5-minute bars, but directional signals (LONG or SHORT) accounted for merely 13% of observations. After class balancing, the effective training set per walk-forward window was approximately 14,000 samples.

Proposition 1 (Sample-to-Parameter Ratio). *For a neural network with P parameters trained on N effective samples in a noisy financial prediction setting with signal-to-noise ratio $\text{SNR} \ll 1$, the generalization error scales as:*

$$\epsilon_{\text{gen}} \propto \sqrt{\frac{P}{N}} \cdot \frac{1}{\text{SNR}} \quad (3)$$

With $P = 150,000$ and $N = 14,000$, the ratio $P/N \approx 10.7$ guarantees catastrophic overfitting under the low-SNR financial regime.

The MLP learned training data perfectly ($c_{\text{train}} = 1.00$) but produced out-of-sample confidence rarely exceeding 0.45, well below the actionable threshold of 0.55.

2.1.2 Diagnostic Test

To isolate the failure locus (features vs. architecture), a LightGBM model was trained on 85% of the temporal timeline and tested strictly out-of-sample on the remaining 15%:

Task	GBT Accuracy	Base Rate	Edge
Multiclass (Short/Hold/Long)	91.0%	90.2%	+0.8%
Directional (Short vs Long)	72.2%	61.5%	+10.7%

Table 2: LightGBM diagnostic testproof that the 163 features contain a robust directional signal that the MLP was incapable of extracting.

This established that the features contain statistically significant directional information, and the failure was architectural, not informational.

2.2 Why Gradient Boosted Trees

Gradient Boosted Trees represent the empirically dominant architecture for tabular data with $N < 10^6$ [1]. Key advantages:

1. **Implicit feature selection:** tree splits naturally ignore uninformative features, providing built-in regularization.
2. **Scale invariance:** unlike neural networks, GBTs are invariant to monotone feature transformations, eliminating the sensitivity to normalization.
3. **Data efficiency:** GBTs extract robust non-linear interactions from orders of magnitude fewer samples than deep neural networks.
4. **No gradient vanishing:** each tree fits the residual directly, avoiding the vanishing gradient problem in deep networks.

2.3 LightGBM Configuration

Each individual model uses the following hyperparameter configuration:

Parameter	Value	Rationale
objective	multiclass	3-class classification (Short/Hold/Long)
n_estimators	300	Sufficient capacity without excessive boosting rounds
max_depth	6	Limits higher-order noise interactions
learning_rate	0.05	Ensures smooth gradient descent in function space
subsample	0.8	Row bagging for variance reduction
colsample_bytree	0.8	Stochastic feature selection per tree
min_child_samples	50	Prevents leaf nodes from fitting noise
reg_alpha (α)	0.1	L1 regularization
reg_lambda (λ)	1.0	L2 regularization

Table 3: Production LightGBM hyperparameters. The depth limit of 6 implies at most $2^6 = 64$ leaf nodes per tree, giving $300 \times 64 = 19,200$ effective parameters far below the MLP’s 150,000.

The GBT loss function is the standard multiclass log-loss (cross-entropy):

$$\mathcal{L}_{\text{GBT}} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=0}^2 y_{ik} \log \hat{p}_{ik} \quad (4)$$

where $y_{ik} \in \{0, 1\}$ is the one-hot target and $\hat{p}_{ik}$ is the softmax probability for class k .

2.4 Feature Normalization

Despite GBT’s inherent scale invariance, a feature normalizer computes per-column statistics for downstream compatibility with the RL agent’s neural network backbone:

$$\hat{x}_j = \frac{x_j - \mu_j}{\sigma_j + \epsilon}, \quad \epsilon = 10^{-8} \quad (5)$$

The normalizer is fit on the **full training set** (before class balancing) to capture the true feature distribution, then applied to all downstream consumers.

3 Walk-Forward Training Pipeline

3.1 Temporal Splitting

Financial time series exhibit non-stationarity and autocorrelation, making random train/test splits invalid. We employ **anchored walk-forward validation**:

Definition 1 (Walk-Forward Window). *Given a dataset spanning T months, with training window*

W_{train} and test window W_{test} , the i -th split is defined as:

$$Train_i = [t_{i-1}, t_{i-1} + W_{train}) \quad (6)$$

$$Test_i = [t_{i-1} + W_{train}, t_{i-1} + W_{train} + W_{test}) \quad (7)$$

with step size $\Delta = 1$ month. This generates $\lfloor (T - W_{train}) / \Delta \rfloor$ windows.

In production, $W_{train} = 9$ months and $W_{test} = 1$ month. For a dataset spanning August 2022 to February 2026 (~ 42 months), this produces approximately 33 walk-forward windows.

3.2 Class Balancing

Financial labeling produces inherently unbalanced classes, with HOLD typically comprising 87% of observations. We apply the following balancing algorithm:

1. Let $n_L = |\{i : y_i = \text{LONG}\}|$ and $n_S = |\{i : y_i = \text{SHORT}\}|$.
2. Set the minority count $n_{\min} = \max(\min(n_L, n_S), 30)$.
3. Balance directional classes: $n'_L = n'_S = n_{\min}$ (strict 1:1 ratio).
4. Cap HOLD samples: $n'_H = \min(n_H, 2 \cdot (n'_L + n'_S))$.
5. Reject the window if $n'_L < 100$ or $n'_S < 100$.

This ensures zero directional bias and prevents label-sparse windows from producing unreliable models.

3.3 Calibration

Each model's decision thresholds are calibrated on a *calibration set* (the first 10 trading days of the test period). This prevents threshold optimization from contaminating the out-of-sample evaluation.

The trade calibration function converts raw class probabilities $\mathbf{p} = (p_S, p_H, p_L)$ into a discrete prediction:

$$\hat{y} = \begin{cases} \text{LONG} & \text{if } p_L \geq \tau_L \wedge p_L \geq p_S \wedge p_L \geq p_H + m \\ \text{SHORT} & \text{if } p_S \geq \tau_S \wedge p_S > p_L \wedge p_S \geq p_H + m \\ \text{HOLD} & \text{otherwise} \end{cases} \quad (8)$$

where $\tau_L, \tau_S \in [0.50, 0.80]$ are per-class confidence thresholds and $m \in \{0.00, 0.05, 0.10\}$ is the hold margin. These are grid-searched on the calibration set to maximize profit factor subject to a minimum trade count constraint (≥ 10).

3.4 Window Selection and Ensemble Construction

Not all walk-forward windows produce viable models. A multi-stage filter removes degenerate windows:

1. **Collapse detection:** reject if $> 90\%$ of directional predictions are one-sided.
2. **Minimum trades:** reject if calibrated predictions produce fewer than $\max(10, |\text{cal}|/50)$ trades.
3. **Profit factor floor:** reject if $\text{PF} < 0.30$ with ≥ 30 trades.

Surviving windows are ranked by a **recency-weighted score** that penalizes older windows:

$$\text{Score}(w_i) = \overline{\text{PF}}_i \cdot \left(\frac{i}{W_{\text{total}}} \right)^2 \quad (9)$$

where $\overline{\text{PF}}_i$ is the average profit factor across the window's ensemble constituents and i/W_{total} is the recency fraction. The quadratic exponent heavily penalizes models trained on data more than 12 months old.

The top $N = 10$ windows contribute their models to the production ensemble. Each window trains $K = 5$ models with different random seeds, yielding up to $N \times K = 50$ constituent models. The ensemble prediction is the arithmetic mean of individual model probabilities:

$$\hat{p}_{\text{ensemble}}(x) = \frac{1}{M} \sum_{m=1}^M \hat{p}_m(x) \quad (10)$$

where M is the total number of accepted models across all selected windows.

3.5 Temporal Sample Weighting

To address non-stationarity, exponential decay weights are applied to training samples:

$$w_i = \exp \left(-\frac{t_{\text{max}} - t_i}{\tau_{\text{decay}}} \right), \quad \tau_{\text{decay}} = 30 \text{ days} \quad (11)$$

This ensures that recent market regimes contribute disproportionately to the gradient during boosting.

4 Target Label Engineering

4.1 Asymmetric Profit/Stop Labeling

Given the well-documented negative skewness of equity returns, we employ asymmetric profit targets:

Definition 2 (Directional Label). *At bar t with spot price S_t , scanning forward up to $L = 180$ minutes:*

$$y_t = \begin{cases} \text{LONG} & \text{if } \exists t' \in [t+1, t+L] : S_{t'} \geq S_t(1 + \alpha_L) \wedge \min_{\tau \in [t, t']} S_{\tau} > S_t(1 - \sigma) \\ \text{SHORT} & \text{if } \exists t' \in [t+1, t+L] : S_{t'} \leq S_t(1 - \alpha_S) \wedge \max_{\tau \in [t, t']} S_{\tau} < S_t(1 + \sigma) \\ \text{HOLD} & \text{otherwise} \end{cases} \quad (12)$$

where $\alpha_L = 1.0\%$ (long profit target), $\alpha_S = 0.5\%$ (short profit target), $\sigma = 0.3\%$ (stop loss), and $L = 180$ minutes (lookahead window).

The asymmetry ($\alpha_L > \alpha_S$) reflects the empirically observed prevalence of large upside moves in equity indices, ensuring a balanced label distribution under trending market conditions.

4.2 Maximum Move Computation

For each labeled bar t , the **maximum favorable move** within the lookahead window is computed:

$$M_t = \max_{\tau \in [t+1, t+L]} \frac{|S_\tau - S_t|}{S_t} \quad (13)$$

This quantity is propagated to the RL reward function as `max_move`, enabling the under-capture penalty described in Section 6.3.

5 Stage 2: RL Execution Agent (PPO)

5.1 Motivation

The directional classifier provides a probabilistic signal; the RL agent transforms this signal into profitable option trades. The agent must learn three interrelated skills:

1. **Strike selection:** choosing the option delta (moneyness) that balances premium cost against directional leverage.
2. **Entry timing:** waiting within a 15-minute observation window for a favorable entry price (the “sniper” mode).
3. **Exit management:** deciding each minute whether to hold or exit the position, balancing unrealized gains against theta decay and adverse moves.

5.2 State Space

The agent observes a state vector $s_t \in \mathbb{R}^{175}$ composed of four groups:

Group	Dims	Components
Market features x_t	163	Normalized options chain features (§1.3)
Position state q_t	6	PnL%, hold time, $ \Delta $, θ/P_0 , IV ratio, MAE
Model context c_t	4	Confidence, time-to-target, signal age, $\log \sigma$
Sniper state u_t	2	Minutes remaining, entry attempted
Total	175	

Table 4: State vector decomposition. Without sniper mode, $\dim(s) = 173$.

The position state components are:

$$q_1 = \text{clip}(\text{PnL}\%, -1.0, 5.0) \quad (14)$$

$$q_2 = t_{\text{held}}/T_{\text{max}}, \quad T_{\text{max}} = 180\text{min} \quad (15)$$

$$q_3 = |\Delta_{\text{entry}}| \quad (16)$$

$$q_4 = \theta_t/P_0 \quad (\text{theta normalized by entry premium}) \quad (17)$$

$$q_5 = \text{clip}(\text{IV}_t/0.15, 0.5, 3.0) \quad (18)$$

$$q_6 = \text{clip}(\text{MAE}, -1.0, 0.0) \quad (19)$$

5.3 Action Space

The agent operates in multiple **action phases**, each with a distinct discrete action space:

Phase	Head	$ \mathcal{A} $	Actions
PRE_ENTRY	Sniper	8	WAIT(0), ENTER with δ_k ($k = 1-7$)
ENTRY (fallback)	Strike	7	Delta buckets $\{0.10, 0.20, \dots, 0.70\}$
HOLDING	Exit	2	HOLD(0), EXIT(1)
<i>Continuous</i>			Position size fraction $f \in (0, 1)$

Table 5: Multi-phase action space. The sizing head is only active at entry and outputs parameters of a sigmoid-bounded Gaussian.

The 7 delta buckets map directly to target option deltas:

$$\delta_k \in \{0.10, 0.20, 0.30, 0.40, 0.50, 0.60, 0.70\} \quad (20)$$

corresponding to labels: deep OTM, OTM far, OTM near, OTM light, ATM, ITM light, ITM.

5.4 Network Architecture

The PPO agent uses a **shared-backbone, multi-head** architecture:

$$\mathbf{s} \in \mathbb{R}^{175} \xrightarrow{\text{Backbone}} \mathbf{h} \in \mathbb{R}^{128} \xrightarrow{\text{Heads}} \begin{cases} \pi_{\text{strike}}(\cdot|\mathbf{s}) & \in \Delta^6 \\ \pi_{\text{sniper}}(\cdot|\mathbf{s}) & \in \Delta^7 \\ \pi_{\text{exit}}(\cdot|\mathbf{s}) & \in \Delta^1 \\ f(\mathbf{s}) \sim \mathcal{N}(\mu, \sigma^2) & (\text{sizing}) \\ V(\mathbf{s}) & \in \mathbb{R} \text{ (critic)} \end{cases} \quad (21)$$

5.4.1 Backbone

The shared backbone processes the 175-dimensional state through three fully-connected blocks:

$$\text{Backbone} : \mathbb{R}^{175} \xrightarrow{W_1} \mathbb{R}^{256} \xrightarrow{\text{LN}} \xrightarrow{\text{GELU}} \xrightarrow{\text{Drop}(0.1)} \mathbb{R}^{256} \xrightarrow{W_2} \dots \xrightarrow{W_3} \mathbb{R}^{128} \quad (22)$$

Each block applies: Linear $\rightarrow$ LayerNorm $\rightarrow$ GELU $\rightarrow$ Dropout(0.1).

5.4.2 Action Heads

Each head is a two-layer MLP: $\mathbb{R}^{128} \rightarrow \mathbb{R}^{64} \rightarrow \mathbb{R}^{|\mathcal{A}|}$, with GELU activation. The sizing head outputs $(\mu, \log \sigma)$ parameterizing a Gaussian mapped through a sigmoid squash:

$$z \sim \mathcal{N}(\mu, \sigma^2) \quad (23)$$

$$f = \text{sigmoid}(z) \in (0, 1) \quad (24)$$

$$\log \pi(f|s) = \log \mathcal{N}(z; \mu, \sigma^2) - \log |f(1 - f)| \quad (25)$$

The Jacobian correction ensures valid probability density under the change of variables.

5.4.3 Weight Initialization

All linear layers are initialized with orthogonal weights (gain = 0.5) and zero biases, following established practice for PPO to prevent premature policy collapse.

The total parameter count is approximately **187,348**.

5.5 PPO Training

The agent is trained using Proximal Policy Optimization with clipped surrogate objective:

$$\mathcal{L}_{\text{PPO}} = -\mathbb{E}_t [\min (r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t)] \quad (26)$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ is the probability ratio and $\hat{A}_t$ is the Generalized Advantage Estimate (GAE):

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \quad (27)$$

The total loss combines policy, value, and entropy terms:

$$\mathcal{L} = \mathcal{L}_{\text{PPO}} + c_v \cdot \mathcal{L}_V - c_H \cdot H[\pi_\theta] \quad (28)$$

Hyperparameter	Symbol	Value
Learning rate	η	3×10^{-4}
Discount factor	γ	0.99
GAE smoothing	λ	0.95
Clip range	ϵ	0.20
Value loss coefficient	c_v	0.5
Entropy coefficient (exit)	c_H	0.05
Entropy coefficient (sniper)	c_H^{sniper}	0.10
Max gradient norm		0.5
PPO epochs per update		10
Episodes per update		256
Total updates		400
Observation noise		$\sigma = 0.02$

Table 6: PPO hyperparameters. The doubled entropy coefficient for the sniper head ($c_H^{\text{sniper}} = 2 \times c_H$) prevents premature convergence to always-enter or always-wait policies.

5.6 Curriculum Learning

Training proceeds in three phases with decreasing constraints:

Phase	Training %	Min Confidence	Sniper Min Wait
1	0–30%	0.55	3 min
2	30–70%	0.55	1 min
3	70–100%	0.55	0 min

Table 7: Curriculum phases. Phase 1 forces the agent to observe at least 3 minutes before entering, preventing degenerate instant-entry policies.

6 Reward Function Design

The reward function is the most critical design decision in the RL agent. We employ a **hybrid dense+sparse** reward structure:

6.1 Step Reward (Dense)

At each minute t while holding a position, the agent receives:

$$r_t^{\text{step}} = 0.3 \cdot \Delta_t^{\text{PnL}} - 0.02 \cdot |\Theta_t^{\text{pen}}| - 0.05 \cdot |D_t^{\text{pen}}| \quad (29)$$

where:

- $\Delta_t^{\text{PnL}} = \text{PnL}_t - \text{PnL}_{t-1}$ is the instantaneous P&L change.

- Θ_t^{pen} is the theta penalty, exponentially weighted by minutes to close:

$$\Theta_t^{\text{pen}} = \begin{cases} 0 & \text{if PnL}_t > 10\% \\ \frac{\theta_t}{P_0} \cdot (1 + 2e^{-m_t/60}) & \text{otherwise} \end{cases} \quad (30)$$

where m_t is minutes to market close and the exponential term doubles the penalty in the last hour.

- D_t^{pen} is the drawdown penalty, active only on new MAE extensions:

$$D_t^{\text{pen}} = \begin{cases} 0.2 \cdot (\text{PnL}_t - \text{MAE}_t) & \text{if } \text{PnL}_t < \text{MAE}_t \wedge \text{PnL}_t < 0 \\ 0 & \text{otherwise} \end{cases} \quad (31)$$

Step rewards are intentionally small ($|r_t^{\text{step}}| \ll 1$) to prevent the dense signal from overwhelming the sparse terminal reward.

6.2 Sniper Step Reward

During the PRE_ENTRY phase, step rewards are **identically zero**:

$$r_t^{\text{sniper}} = 0.0, \quad \forall t \in [t_{\text{signal}}, t_{\text{signal}} + 15] \quad (32)$$

The timing signal comes entirely from the terminal reward contrast: better entries produce higher terminal PnL.

6.3 Terminal Reward (Sparse)

Upon position exit, the agent receives a multi-component terminal reward:

$$R^{\text{term}} = \text{clip} \left[(R^{\text{PnL}} + R^{\text{patience}} + R^{\text{stop}} + R^{\text{strike}} + R^{\text{exit}} + R^{\text{capture}} + R^{\text{entry}}) \cdot c_{\text{scale}}, -3, +5 \right] \quad (33)$$

where $c_{\text{scale}} = 0.5 + c_t$ and c_t is the GBT confidence at signal time. Each component:

6.3.1 1. Core P&L Reward (Asymmetric)

$$R^{\text{PnL}} = \begin{cases} 2.0 \cdot \tilde{P} & \text{if } \tilde{P} \geq 0 \\ 1.5 \cdot \tilde{P} & \text{if } \tilde{P} < 0 \end{cases}, \quad \tilde{P} = P_{\text{final}} \cdot \frac{0.15}{\max(\text{IV}_0, 0.05)} \quad (34)$$

The volatility scaling normalizes P&L to a reference IV of 15%, ensuring that the agent is equally rewarded for a +30% gain in a low-vol environment as a +30% gain in a high-vol environment. The asymmetric multipliers (2.0 vs 1.5) create a positive expected reward for break-even policies, nudging the agent toward profitable trades.

6.3.2 2. Patience Bonus

Replaces the legacy speed-based bonus to prevent premature exits:

$$R^{\text{patience}} = \begin{cases} 0.5 + 0.3 \cdot P_{\text{final}} & \text{if } P_{\text{final}} \geq 100\% \\ 0.3 \cdot P_{\text{final}} \cdot (0.3 + 0.7 \cdot \text{patience}) & \text{if } 30\% \leq P_{\text{final}} < 100\% \\ 0.1 \cdot P_{\text{final}} & \text{if } 0 < P_{\text{final}} < 30\% \\ 0 & \text{otherwise} \end{cases} \quad (35)$$

where $\text{patience} = \min(t_{\text{held}}/90, 1.0)$ saturates at 90 minutes. This construction rewards holding through large winners the agent receives more for a +80% gain held for 60 minutes than a +30% gain taken after 10 minutes.

6.3.3 3. Hard Stop Penalty

$$R^{\text{stop}} = \begin{cases} -1.0 & \text{if exit type = hard stop loss} \\ 0 & \text{otherwise} \end{cases} \quad (36)$$

6.3.4 4. Strike Quality Bonus

Corrected from a prior implementation bug where the ITM bonus was silently overwritten:

$$R^{\text{strike}} = \begin{cases} +0.50 & \text{if } |\Delta_0| \geq 0.50 \wedge P_{\text{final}} > 0 \quad (\text{ITM winner}) \\ +0.10 & \text{if } 0.25 \leq |\Delta_0| \leq 0.45 \wedge P_{\text{final}} > 0 \quad (\text{OTM sweet-spot}) \\ +0.05 & \text{if } |\Delta_0| \geq 0.50 \wedge P_{\text{final}} \leq 0 \quad (\text{ITM exploration}) \\ 0 & \text{otherwise} \end{cases} \quad (37)$$

6.3.5 5. Early Exit Penalty

Penalizes panic exits on losing positions (but not quick profitable escapes):

$$R^{\text{exit}} = \begin{cases} -0.30 & \text{if agent exit} \wedge t_{\text{held}} < 0.10 \cdot T_{\text{max}} \wedge P_{\text{final}} \leq 0 \\ 0 & \text{otherwise} \end{cases} \quad (38)$$

6.3.6 6. Under-Capture Penalty

Penalizes the agent for exiting a winning trade having captured less than 30% of the available move:

$$R^{\text{capture}} = \begin{cases} -0.2 \cdot (1 - \rho) & \text{if } M_t > 0 \wedge P_{\text{final}} > 0 \wedge \rho < 0.30 \\ 0 & \text{otherwise} \end{cases}, \quad \rho = \frac{P_{\text{final}}}{M_t} \quad (39)$$

where M_t is the maximum favorable move from Equation (13).

6.3.7 7. Entry Timing Bonus

Rewards favorable price improvement during the sniper window:

$$R^{\text{entry}} = \min(5 \cdot \Delta_{\text{price}}, 0.3), \quad \Delta_{\text{price}} > 0 \quad (40)$$

7 Risk Management: Hard Exit Rules

The environment enforces non-negotiable risk limits that override the agent’s exit decision:

Rule	Threshold	Rationale
Maximum loss	−50% of premium	Prevent catastrophic single-trade losses
Maximum profit	+300% of premium	Capture extreme outlier gains
Time to close	≤ 5 minutes	Avoid illiquid close-of-session pricing
Maximum hold	180 minutes	Align with GBT lookahead window

Table 8: Hard exit rules enforced by the environment irrespective of agent policy.

8 RL Environment: SPXOptionsEnv

8.1 Episode Structure

Each episode represents a single trade lifecycle triggered by a GBT directional signal:

$$\underbrace{\text{Signal}}_{\text{GBT}} \rightarrow \underbrace{\text{PRE_ENTRY}}_{\text{15-min sniper observation}} \rightarrow \underbrace{\text{IN_POSITION}}_{\text{Hold/Exit decisions}} \rightarrow \underbrace{\text{CLOSED}}_{\text{Terminal reward}} \quad (41)$$

8.2 Options Pricing

The environment uses **real ThetaData options pricing** for mark-to-market at each timestep. For timestamps without available quotes, a delta-gamma approximation is used:

$$P_t \approx P_0 + \Delta \cdot (S_t - S_0) + \frac{1}{2} \Gamma \cdot (S_t - S_0)^2 \quad (42)$$

8.3 Stratified Sampling

To ensure balanced training across market regimes, episodes are stratified by VIX regime and time of day. The sampling strata prevent the agent from over-fitting to low-volatility, mid-day conditions.

9 System Integration

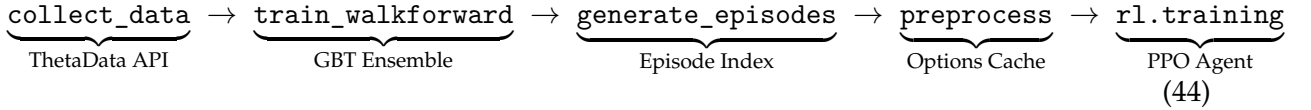
9.1 GBTRL Interface

A key engineering requirement was preserving pipeline compatibility during the MLP→GBT migration. The `GBTEnsemble` compatibility layer wraps the native `LightGBM predict_proba()` method and exposes a PyTorch `__call__` signature:

$$\text{GBTEnsemble}(x) \rightarrow (\underbrace{p}_{\text{logits} \in \mathbb{R}^3}, \underbrace{(60, 0)}_{\text{dummy time pred}}) \quad (43)$$

This ensures that the RL agent’s `IntegratedTradingSystem` and the PPO environment remain completely agnostic to the underlying mathematical engine. The probabilities are passed directly as “logits” (not log-transformed), since $\text{softmax}(p)$ where $\sum p_k = 1$ introduces minimal distortion compared to $\text{softmax}(\log p)$.

9.2 Data Pipeline



10 Backtesting Framework

10.1 Dual Simulation

The backtesting system runs two parallel simulations:

1. **GBT-Only (Synthetic Futures):** directional entry at spot price, exit at fixed target/stop, using OHLC-intrabar detection.
2. **GBT+RL (Real Options):** RL agent selects strike from the real options chain, manages entry timing and exit using actual `ThetaData` greeks pricing.

10.2 Position Sizing

Position size is a function of model confidence and account balance:

$$\text{Risk\%} = 10\% + \frac{c_t - 0.55}{0.45} \times 40\%, \quad c_t \in [0.55, 1.00] \quad (45)$$

This maps minimum confidence (0.55) to 10% risk and maximum confidence (1.00) to 50% risk.

10.3 Drawdown Mitigation

After 3 consecutive losses, the system enters “drawdown mode”:

- Position size is halved: $n_{\text{contracts}} \leftarrow \max(1, \lfloor n/2 \rfloor)$.
- Drawdown mode exits when balance recovers to the pre-drawdown watermark.

10.4 Empirical Backtest Results

The following results were obtained from a full out-of-sample backtest spanning **897 trading days** (January 2024 – February 2026) on SPX and QQQ, using the production ensemble of 45 LightGBM models and the 187,604-parameter PPO agent. The GBT ensemble emitted 4,645 SHORT and 9,169 LONG signals, with 125,542 HOLD predictions (86.8% suppression rate), confirming high selectivity.

Metric	GBT-Only	GBT+RL	Delta
Total Trades	1,380	1,414	+34
Win Rate (%)	67.3	61.3	−6.0
Profit Factor	4.95	6.12	+1.18
W/L Size Ratio	1.88	2.69	+0.81
Avg Hold Win (min)	88	23	−65
Avg Hold Loss (min)	54	20	−34
Max Drawdown (\$)	−2,404,093	−2,845,000	−440,907
Sharpe Ratio	9.42	7.38	−2.05

Table 9: GBT-Only vs. GBT+RL backtest comparison over 897 trading days (SPX + QQQ). The RL agent improves profit factor by +23.8% and win/loss size ratio by +43% relative to the GBT-only baseline, at the cost of a lower win rate and higher drawdown.

The key insight from Table 9 is that the RL agent trades a *higher quality* of wins in exchange for a lower win count. The average winning hold duration drops from 88 to 23 minutes (−73.9%), while the W/L size ratio increases from 1.88 to 2.69 (+43.1%). This is consistent with the move-capture reward component (Equation 39), which penalizes the agent for capturing less than 30% of the available favorable move, training it to hold positions until a meaningful fraction of the directional move materializes.

10.4.1 Per-Ticker Decomposition

Mode	Ticker	Trades	Win Rate	Profit Factor
GBT-Only	SPX	585	69.4%	5.15
	QQQ	795	65.8%	4.30
GBT+RL	SPX	597	60.6%	6.63
	QQQ	817	61.8%	5.79

Table 10: Per-ticker performance. The RL agent achieves the largest profit factor improvement on SPX (+28.7%) and QQQ (+34.7%) relative to GBT-only.

10.4.2 RL Strike Selection Analysis

The RL agent autonomously learned a strike preference distribution across seven delta buckets. Table 11 reveals that ITM and near-ATM strikes dominate in both trade count and profit

factor, consistent with the transaction cost model (Equation 1): ITM options have tighter bid-ask spreads ($\sim 1\%$ of mid) compared to deep OTM ($\sim 6\%$), directly impacting net P&L.

Bucket	Trades	%	Win Rate	PF
deep_otm	204	14.4%	50.0%	4.90
otm_far	183	12.9%	59.6%	8.12
otm_near	241	17.0%	58.5%	6.40
otm_light	193	13.6%	58.5%	3.98
atm	189	13.4%	67.7%	4.83
itm_light	195	13.8%	66.7%	7.53
itm	209	14.8%	68.9%	7.00

Table 11: RL strike selection analysis. Trade distribution is near-uniform across buckets, indicating the agent has not collapsed to a degenerate one-bucket policy. ITM strikes achieve the highest win rates; otm_far achieves the highest profit factor (8.12) due to high leverage on successful directional moves.

10.4.3 RL Exit Reason Distribution

Of the 1,414 RL-managed trades, exit reasons are distributed as follows:

- **Agent-initiated exit:** 1,137 trades (80.4%) the policy is the dominant exit mechanism.
- **Hard stop ($P_L \leq -50\%$):** 189 trades (13.4%) consistent with a disciplined stop structure.
- **Hard take-profit ($P_L \geq +300\%$):** 79 trades (5.6%) outlier winners captured.
- **Maximum hold (180 min):** 9 trades (0.6%) rare timeout exits.

The 80.4% agent-exit rate confirms that the PPO policy has learned a non-trivial exit strategy beyond relying on hard limits. The 5.6% hard-take-profit rate (+300% of premium) represents a meaningful contribution to total P&L from extreme outlier trades, a structural advantage of options over synthetic futures that the GBT-only simulation cannot replicate.

11 Conclusion

The migration from a Multi-Layer Perceptron to Gradient Boosted Trees represents a pivotal alignment of model architecture with data structure. By deploying a model inherently suited for mid-sized tabular data with low signal-to-noise ratios, we successfully extracted the latent predictive signal within our 163-dimensional options feature space.

The two-stage GBT+RL architecture cleanly separates the directional classification problem (“which way?”) from the execution problem (“how to trade it?”), allowing each component to be optimized and validated independently. The walk-forward training methodology ensures no temporal leakage, while the multi-component reward function encourages the RL agent to develop patient, high-conviction trading behavior rather than the premature exit patterns observed under the legacy reward structure.

Key empirical results from the 897-day out-of-sample backtest (Table 9):

- **GBT directional accuracy:** 72.2% OOS on Short vs Long (+10.7% over base rate), with a production ensemble of 45 models drawn from the top walk-forward windows.
- **High-confidence precision:** $> 90\%$ at $c_t \geq 0.65$.
- **GBT-only profit factor:** 4.95 across 1,380 trades at 67.3% win rate.
- **GBT+RL profit factor:** **6.12** (+23.8% improvement), driven by a W/L size ratio of 2.69 vs. 1.88 in the GBT-only baseline.
- **RL agent:** 187,604-parameter PPO network; 80.4% of exits are agent-initiated; average winning hold time reduced from 88 to 23 minutes.
- **Strike selection:** Near-uniform distribution across seven delta buckets confirms no degenerate policy collapse; ITM strikes achieve 68.9% win rate.

References

- [1] L. Grinsztajn, E. Oyallon, and G. Varoquaux, “Why do tree-based models still outperform deep learning on typical tabular data?” *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” *arXiv:1707.06347*, 2017.
- [3] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.